

TaskFlow

Görev ve Proje Yönetim Sistemi — Proje Tanıtım Dokümanı

Proje adı	TaskFlow — Görev ve Proje Yönetim Sistemi
Yazar	Mehmet Özdemir
Teknolojiler	Node.js, Express.js, JSON dosya kalıcılığı, Vanilla HTML/CSS/JavaScript

Amaç ve Ele Alınan Problem

Küçük yazılım ekipleri, görevleri genellikle dağınık notlar veya sohbet uygulamaları üzerinden takip eder; bu da görevlerin kimde olduğunu, hangi aşamada bulunduğunu ve öncelik sırasını izlemeyi zorlaştırır. TaskFlow, bu ihtiyaca cevap veren sade bir REST API ve bu API'yi kullanan bir web arayüzüdür: bir ekip, görev oluşturabilir, bir çalışana atayabilir, durumunu (`pending`, `in-progress`, `completed`) ve önceliğini (`low`, `medium`, `high`) güncelleyebilir, arama/filtre/sayfalama ile listeleri yönetebilir ve anlık özet raporlarla ekip yükünü görebilir.

Teknik Mimari

Proje, Node.js 18+ üzerinde çalışan bir Express.js uygulamasıdır. Uygulama katmanları net biçimde ayrılmıştır: `routes` yalnızca HTTP yollarını tanımlar, `controllers` iş mantığını yürütür, `middleware` loglama/doğrulama/hata yönetimini üstlenir ve `utils/file.js` tüm dosya erişimini tek bir yerde toplar. Route dosyaları doğrudan dosya sistemine erişmez. Frontend, aynı Express uygulamasından `public/` klasörüyle statik olarak sunulur; ayrı bir build sistemi veya framework kullanılmaz.

Backend ve Kalıcılık Yaklaşımı

Veri katmanı bilinçli olarak bir veritabanı yerine `data/tasks.json` dosyasını kullanır; bu, kursun kapsamına uygun, açıklanabilir ve sade bir tercihtir. `readTasks()` / `writeTasks()` fonksiyonları JSON'u okunabilir biçimde (2 boşluk girinti) saklar; her başarılı oluşturma, güncelleme ve silme işleminden sonra dosyaya yazılır, liste/arama/rapor istekleri ise dosyayı hiçbir zaman değiştirmez. Beş zorunlu CRUD endpoint'ine ek olarak durum/öncelik filtreleme, anahtar kelime arama, çalışana göre listeleme, sayfalama, oluşturulma tarihine göre sıralama ve üç raporlama endpoint'i (tamamlanan, bekleyen, özet) uygulanmıştır. Tüm POST/PUT istekleri, hataları tek bir listede toplayan bir doğrulama katmanından geçer.

Frontend ve Responsive Tasarım

Arayüz, tüm verisini `fetch()` ile API'den alan responsive bir tek sayfa panelidir: istatistik kartları, filtre/arama araç çubuğu, görev listesi, oluşturma/düzenleme modalı ve silme onay modalı içerir. Erişilebilirlik gözetilerek görünür etiketler, klavye ile kullanılabilir kontroller, odak/'focus-visible' stili, atlama bağlantısı ve `aria-live` duyuruları eklenmiştir. Arayüz 1024px, 720px ve 480px kesme noktalarında masaüstünden mobile kadar yatay taşma olmadan çalışır.

Doğrulama ve Hata Yönetimi

Sunucu, tutarlı bir cevap zarfı kullanır: başarılı işlemlerde `{success, message, data}` veya `{success, data, meta}`, doğrulama hatalarında `{success:false, message:"Validation failed", errors:[...]}`. Geçersiz durum/öncelik değerleri, eksik zorunlu alanlar, geçersiz sayfalama/sıralama parametreleri ve bozuk JSON gövdesi her zaman `400`; olmayan görev veya bilinmeyen endpoint her zaman `404` döndürür. Hiçbir hata cevabında stack trace veya sunucu dosya yolu sızdırılmaz.

Test ve Postman Doğrulaması

Proje üç bağımsız doğrulama katmanından geçmiştir: geliştirme sürecinde Node.js ve `curl` ile yürütülen otomatik HTTP kontrolleri (CRUD, doğrulama, gelişmiş listeleme, raporlar, dosya kalıcılığı); gerçek Chrome tarayıcısında yapılan manuel arayüz doğrulaması (masaüstü/mobil düzen, oluşturma/düzenleme/silme akışları, Türkçe karakter render'ı); ve Postman masaüstü uygulamasında gerçekten import edilip çalıştırılan bir Postman Collection v2.1 koşumu. Bu son adımdan CRUD, doğrulama hataları, gelişmiş sorgular ve raporları kapsayan 15 gerçek ekran görüntüsü elde edilmiştir.

Sonuç

TaskFlow, sade ve açıklanabilir bir teknoloji seçimiyle (Node.js, Express, JSON dosya kalıcılığı, framework'süz frontend) zorunlu CRUD gereksinimlerini ve seçilen gelişmiş özellikleri (filtreleme, arama, sayfalama, sıralama, raporlama) eksiksiz biçimde karşılayan, otomatik ve manuel olarak doğrulanmış bir uçtan uca örnek projedir.